Advanced Data Management, chapters 1–6: the how, the what, the syntax

blue = topic, green = procedure, red ! = the mistake that costs points, [FS] = on the exam's formula sheet, not repeated. B pages, N rows, r rows/page, M buffer pages for an operator (`work_mem`/8 KB), h height, F fanout, L entries/leaf, s selectivity, d distinct values, k matches, W working set.

1 Architecture and the life of a query

Five subsystems: session manager (one worker per connection) ▹ query processor (parser, binder, rewriter, planner, executor) ▹ storage manager (heap files, indexes, buffer pool) ▹ transaction manager (locks, log, recovery) ▹ catalog (schema and statistics: `pg_class`, `pg_stats`).

Symptom → owner: syntax error: parser ▹ "column does not exist", type or permission error: binder, from the catalog ▹ Seq Scan where an index is cheaper, bad join order: planner ▹ estimate 1,000× off: statistics (`ANALYZE`) ▹ low hit ratio: buffer pool ▹ lost update, deadlock: transaction manager ▹ spilled sort: `work_mem`.

Five stages: parse (grammar only) → bind (names, types, permissions) → rewrite (logical: views inlined, subqueries flattened, selection and projection pushed down) → optimise (physical: access path, join algorithm, join order, by cost from statistics) → execute (iterators).

Algebra: $\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$ when p touches R only (push the filter below the join); drop unused columns early; $\bowtie$ is commutative and associative, so the join order is free and must be searched.

Iterator model: every node has `open/next/close`; one row at a time flows up. **Streaming**: Seq/Index Scan, Filter, Limit, Nested Loop, Merge Join, hash *probe*, GroupAggregate on sorted input. **Blocking** (consumes all input before its first output): Sort, Hash (build side), HashAggregate, Materialize. Nothing above a breaker starts before it finishes; a breaker may spill (`Disk`, `Batches`).

SQL evaluation order: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. A SELECT alias works only in ORDER BY; a condition on an aggregate goes in HAVING; every selected column is grouped or aggregated.

OLTP (short transactions, rows by key): row store, B-trees, high hit ratio. **OLAP** (long scans of few columns): column store reads $1/k$ of the bytes for 1 of k columns, but a row write touches k places.

! "The parser catches a misspelled column" (binder). "The executor chooses the scan" (planner). "Hash Join is blocking" (its build child is; the probe streams). Alias in WHERE. Aggregate in WHERE.

2 Storage: count pages, not rows

RAM 100 ns ▹ SSD 100 μs ▹ HDD 10 ms. Disk is 10^3–$10^5\times$ RAM, so cost = pages read + written; CPU matters only when all is cached. Sequential ($\approx 16\,\mu$s per page) beats random (one latency per page).

record = column bytes + 24, rounded up to a multiple of 8 — PostgreSQL tuple header; big values go out of line (TOAST)

$r = \lfloor (8192 - 24)/(\text{record} + 4) \rfloor$ rows per page: 24 B page header, 4 B slot per row, floor

$B = \lceil N/r \rceil$, size = $8B$ KB — compare with `pg_relation_size`; a gap = dead tuples or TOAST

Slotted page: header, slot directory (offset, length) growing down, tuples growing up. Row id = (page, slot) = `ctid`; a tuple can move inside its page without touching any index.

Heap, sorted, hashed file (pages touched): scan all: B, B, B ▹ equality, present: $B/2$ (worst B), $\lceil \log_2 B \rceil$, ≈ 1 ▹ equality, absent: B, $\lceil \log_2 B \rceil$, ≈ 1 ▹ range: B, $\log_2 B$+ pages of the range, B ▹ insert: 1 (append), up to B (shift), 1. Equality → hashed; ranges and order → sorted; append and scan → heap. Real systems: heap + indexes.

PostgreSQL: table = heap file of 8 KB pages. DELETE/UPDATE leave dead tuples (`n_dead_tup`). **VACUUM** frees them inside pages and truncates only empty *trailing* pages, so the file does not shrink after a scattered delete. **VACUUM FULL** and **CLUSTER** rewrite the file (CLUSTER also orders it by an index: correlation → 1). **CPU-bound**: all hits, `read` ≈ 0, second run no faster.

! Counting bytes or rows instead of pages. Forgetting the slot or the floor. "An absent key costs $B/2$" (it is B). "Hashing helps ranges". "VACUUM shrinks the file". Trusting a fast second run (cache).

3 Indexes

An index maps key → `ctid` in its own file; an entry is key +6 B. Height, point lookup, range cost: [FS]. Find h as the smallest h with $F^h \geq N$, typically 3–4; never $\log_2$. Top levels are cached: a lookup is often 1–2 real reads.

B⁺ tree: routing keys inside, all entries in sorted, chained leaves (a range = one descent, then walk right); one page per node; always balanced. **Insert**: descend, insert in the leaf; if full, **split**: lower half stays, upper half moves to a new leaf, chain it, push the first key of the new leaf into the parent; a full parent splits too; a root split adds a level. **Delete**: remove; a leaf under half full borrows from a sibling or merges. **Clustered**: heap in index order, one per table, a range of k rows costs $\lceil k/r \rceil$ pages; **secondary**: any number, k random pages.

Kinds: **B-tree** (default: =, <, BETWEEN, ORDER BY, LIKE 'abc%') ▹ **hash** (= only) ▹ **bitmap index** (one bit vector per value, low cardinality, AND/OR by bit operations; PostgreSQL builds bitmaps on the fly from a B-tree) ▹ **BRIN** (min/max per block range, tiny, physically ordered columns) ▹ **GIN/GiST** (jsonb, text).

Composite `(a,b)`: sorted by `a`, then `b`: only a *leading prefix* descends (`a`; `a AND b`; `a = .. AND b > ..`); `b` alone does not. Equality columns first, the range column last. **Expression** `(lower(email))` serves `WHERE lower(email) = ..` only. **Partial** `WHERE status = 'open'`: small, used only when the query repeats the predicate. **Covering** `INCLUDE (cols)`: index-only scan, `Heap Fetches: 0` after VACUUM.

heap pages touched = $B\,(1-(1-s)^r)$ — matches scattered (correlation ≈ 0); $sr \geq 3 \Rightarrow$ nearly every page

heap pages touched ≈ sB — matches contiguous (correlation ≈ ±1: append-only, after CLUSTER)

plain index scan wins iff $4sN < B \iff s < 1/(4r)$ — scattered case only; random read = 4× sequential

Bitmap scan: collect the row ids, sort by page, read each heap page once in physical order; wins between $1/(4r)$ and a few percent; `Heap Blocks: exact=` counts the pages. **Correlation** (`pg_stats.correlation`) decides which pages formula applies: near ±1 cheap ranges and BRIN; near 0 the index is useless for large s.

Price: every INSERT writes 1 + (number of indexes) random pages; an UPDATE of an indexed column is a delete plus an insert. **Keep**: PK, FK and join columns, selective filters, frequent ORDER BY. **Drop**: `idx_scan = 0`, columns with $s \gg 1/(4r)$ (booleans, a 90 % status).

! $\log_2 N$ for the height. Forgetting the +1 heap page. Counting matching rows, not pages. "I created it, so it is used". (a, b) serving `b` alone. Hash or BRIN for a range. Indexing `name` and querying `lower(name)`. Applying $1/(4r)$ without reading the correlation.

4 Sorting and joins

External merge sort: pass 0 reads M pages at a time, sorts them in memory, writes $\lceil B/M \rceil$ runs; each merge pass merges $M-1$ runs (one buffer is the output). Formulas: [FS]. Procedure for the passes: divide the run count by $M-1$ with a ceiling, repeat until 1, count the divisions. **Doubling** M removes at most one merge pass; it never halves the time. One pass needs $M \geq B$; two passes need $\lceil B/M \rceil \leq M-1$.

PostgreSQL: `Sort Method: quicksort Memory: xkB` fits; `external merge Disk: xkB` spilled, raise `work_mem` to at least the `Disk` figure, per sort, per connection, with `SET LOCAL`; `top-N heapsort` is ORDER BY with LIMIT.

Join costs: [FS] (R outer or build, S inner or probe). **Procedure**: write B_R, B_S, $|R|$, $|S|$, M, d, the indexes and any existing order ▹ put the smaller input outside (nested loop) or as build (hash) ▹ check whether it fits: block NL needs $B_R \leq M-2$, hash needs $B_{\text{build}} \leq M$, otherwise Grace at $3(B_R + B_S)$ ▹ the sort term of a sort-merge is the *full* external sort of that input unless it is already sorted or indexed ▹ compute every candidate, pick the minimum, say when the loser would win.

Which join: **Block NL**: one input fits, any condition (inequality, LIKE); disaster when nothing fits. **Index NL**: tiny outer plus an index on the inner key, about $|R| \times (h+1)$ random reads; disaster when the outer is big (`loops=` huge). **Sort-merge**: inputs already sorted, sorted output wanted, range conditions, little memory; disaster with two full sorts. **Hash**: big equi-join, build fits in `work_mem`; disaster on non-equality, build $\gg M$ (`Batches`), skew. `enable_* = off` adds a 10^{10} penalty, it is not a ban.

Aggregation: **HashAggregate** (few groups, must fit, breaker) vs **Sort + GroupAggregate** (many groups, or input already sorted: streams group by group). DISTINCT is the same choice. Aggregate before sorting when the output must be ordered: sort the groups, not the input.

! $|R|$ rows where B_R pages belongs (that is the simple nested loop). Fan-in M (it is $M-1$); block NL with M (it is $M-2$). Forgetting pass 0. Counting reads only (every pass reads and writes). The large table outside a nested loop. Hash join on <. Sorting the big input to satisfy ORDER BY. "Every aggregate blocks".

5 Optimisation: estimates, costs, search

Statistics (`ANALYZE` samples 30,000 rows): per table `reltuples`, `relpages`; per column `null_frac`, `n_distinct` (negative = a fraction of the rows), `most_common_vals/freqs`, `histogram_bounds` (100 equi-depth buckets of the non-MCV values), `correlation`. Selectivity rules, AND/OR/NOT, join size: [FS]. Defaults when nothing is known: equality 0.5 %, range 1/3.

rows = sN, pages = $\lceil \text{rows}/r \rceil$ (contiguous) or $B(1-(1-s)^r)$ (scattered)

q-error = $\max(\text{est/act},\ \text{act/est})$ — < 2 fine; ≥ 10 a red flag; ≥ 100 the cause of the plan. Under-estimates are the dangerous direction

Errors multiply: a leaf off by x makes every node above it off by at least x; fix the *lowest* wrong node.

Cost model: units are sequential page reads, not milliseconds; prices and the Seq Scan, hash and index NL formulas: [FS]. `cost=start..total`: `start` is the work before the first row (a breaker's whole input); the plan cost is the sum of the nodes, computed bottom-up on *estimated* sizes, so a wrong estimate makes every cost above it wrong.

Access path: plain index iff $s < 1/(4r)$ (scattered) or whenever the matches are contiguous; bitmap up to a few percent; else Seq Scan. **Join order**: n tables give $n!$ left-deep orders; **System-R DP** keeps the best plan per subset, level by level, $2^n - 1$ subsets, plus one plan per *interesting order* (a sort order a later merge join or ORDER BY can reuse); beyond `join_collapse_limit` (8) the written order is frozen, beyond `geqo_threshold` (12) genetic search. **Rule-based** (fixed priorities, no data) vs **cost-based** (statistics; wrong only when they are).

Three failures and their fixes: (1) **stale statistics** (`n_live_tup` ≠ `reltuples`, `last_analyze` old) → `ANALYZE t`. (2) **correlated columns** (`a AND b` where `a` determines `b`: the product under-estimates by $1/s_b$; ANALYZE does not help) → `CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t`. (3) **skew** (one value 90 %, a rare value estimated at $1/d$) → `ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000; ANALYZE t`.

Levers, in order, no hints: statistics → cost constants (`random_page_cost` 1.1 on SSD, `effective_cache_size`) → `work_mem` → `enable_*` for experiments → rewrite the query → `pg_hint_plan` last.

! Adding selectivities for AND. $\min(d)$ or $d_R + d_S$ in the join size. Treating `cost` as ms. Fixing a 1,000× estimate with an index or `enable_hashjoin = off` (symptom, not cause). Reading the inner side's `actual rows` as a total (it is per loop). $n!$ as "the number of plans evaluated" ($2^n - 1$ subsets).

6 Buffer management

Pool = frames + page table (page id → frame). Per frame: **pin count** (operators using it now; > 0 cannot be evicted), **dirty bit** (must be written before the frame is reused), `usage_count`. **Miss path**: pick an unpinned victim, write it if dirty (a second I/O), read the page, pin it. The first touch of any page is a compulsory miss.

EAT = $h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}}$ the miss term dominates unless $h \to 1$: 99 % on HDD ≈ 100 μs = 1,000× RAM

h = hit/(hit + read) from `Buffers:` or `pg_statio_user_tables`; OLTP wants 99.9 %+

`read` = not in `shared_buffers`: served by the OS page cache or the disk, which is why cold and warm runs often differ little. `temp read/written` = a sort or hash spilling. `dirtied/written` = pages modified or flushed by this query.

Why its own pool: it knows the access pattern (scan = use once: ring), the write order (WAL), what is pinned, and its budget; the OS cache knows none of this.

Policies: **OPT** evicts the page used furthest in the future (needs the future: a yardstick) ▸ **FIFO** by load time, a hit changes nothing, cheap, Belady's anomaly (more frames can mean more misses) ▸ **LRU** by last use, a hit moves the page to the front, stack property, but a list move under a lock on every hit ▸ **Clock**: one reference bit, the hand clears bits and evicts the first 0 ▸ **PostgreSQL clock-sweep**: `usage_count` 0–5, +1 per hit, the hand decrements, evict at 0: recency plus frequency, no lock on a hit ▸ **MRU** for a loop larger than the pool.

Sequential flooding: a scan larger than the pool streams use-once pages through every frame and evicts the whole working set W under LRU, clock or FIFO; the next transactions pay W misses; the scan itself gains nothing from caching. **Ring buffers**: a Seq Scan of a table larger than `shared_buffers`/4 cycles through a private ring of 32 frames (256 KB), so W survives if $W \le N - 32$; COPY and CTAS use a 16 MB ring, VACUUM 256 KB; index scans never ring.

Procedure: threshold = `shared_buffers`/4 in pages (128 MB → 16,384 frames → 4,096 pages); below it the table stays resident after one scan, above it every scan `reads` again.

Working set: h climbs steeply until the pool holds the hot set, then plateaus: size for W, not the database; a streaming workload stays at $h \approx 0$ at any size. **Sizing**: `shared_buffers` ≈ 25 % of RAM; more starves the OS cache and `work_mem`, checkpoints get bursty. Measure W and h first.

Dirty pages and recovery: **steal** (a dirty page of an uncommitted transaction may be written) needs *undo*; **force** (commit writes all data pages) is slow and random; **no-force** needs *redo*. Real systems: **steal** + **no-force** + **WAL**: the log record is written before its data page, the log is forced at commit (one sequential append), data pages go later; after a crash redo the committed records, undo the rest. Committed data is safe in the log, not necessarily in the data file.

! "read = disk". "LRU always beats FIFO" (not on a scan larger than the pool). "95 % is a good hit ratio" (compute the EAT). Pin, usage and dirty confused. "No-force loses committed data". "Commit writes the pages" (it writes the log). Tracing LRU as FIFO.

7 Reading a plan

Node line: `(cost=start..total rows=est width=B)` is the planner's guess; `(actual time=first..last rows=act loops=n)` is measured *per loop*: total rows = act × loops, time = last × loops. **Sub-lines**: `Filter` and `Rows Removed by Filter` (tested after reading the row) ▸ `Index Cond` (answered inside the index) ▸ `Recheck Cond` and `Heap Blocks: exact=` (bitmap scan, pages touched) ▸ `Heap Fetches` (index-only; 0 is ideal) ▸ `Hash Cond`, `Merge Cond`, `Join Filter` ▸ `Buckets Batches Memory Usage` (`Batches` > 1 spilled) ▸ `Sort Key`, `Sort Method`, `Memory/Disk` ▸ `Group Key` ▸ `Buffers: shared hit/read/dirtied/written, temp read/written` per node, children included.

Tree: indentation gives the children; the first child is the outer or probe side, the second the inner or build side (under `Hash`). Read bottom-up; execution starts at the leftmost leaf of the deepest breaker.

Five-move method

1 Leaves: scan type per table, filter, est vs act ▸ 2 walk up: join algorithm, order, aggregate, sort ▸ 3 the *lowest* node with a large q-error is the cause ▸ 4 resources: `Buffers` (hit ratio, `read`), `Sort Method`, `Batches`, `loops` on the inner side, `Heap Blocks` ▸ 5 decide: wrong plan from wrong estimates (statistics fix) / right plan but slow (memory, index, cold cache, CPU-bound) / nothing to fix; name the command.

Reconstruct the query: `Seq Scan on t a` = FROM with alias; `Hash Cond`/`Merge Cond` = ON; `Filter`/`Index Cond` = WHERE; `Group Key` = GROUP BY; `Sort Key` = ORDER BY (an aggregate in parentheses = a computed column); `Limit` = LIMIT; `Aggregate` with no key = `count(*)`.

EXPLAIN q: estimates only, nothing runs. **EXPLAIN ANALYZE q**: runs it; actual rows, time, loops, `Sort Method`, `Batches` (DML inside `BEGIN; .. ROLLBACK;`). **(ANALYZE, BUFFERS)**: adds hit/read/temp per node, the only view of the pool.

! Blaming the top node (the cause is the lowest bad estimate). Fixing the scan when the join is the cost: locate the time first.

8 Instruments and SQL to type

Size and pages

```
SELECT reltuples::bigint, relpages FROM
  pg_class WHERE relname = 't'; -- what
  the planner believes
SELECT pg_relation_size('t') / 8192 AS
  pages, pg_size_pretty(
  pg_total_relation_size('t') ); -- real
  file; total = heap + indexes + TOAST
SELECT n_live_tup, n_dead_tup,
  last_analyze FROM pg_stat_user_tables
  WHERE relname = 't'; VACUUM (VERBOSE)
  t; VACUUM FULL t; CLUSTER t USING idx;
```

Statistics

```
SELECT attname, null_frac, n_distinct,
  most_common_vals, most_common_freqs,
  histogram_bounds, correlation FROM
  pg_stats WHERE tablename = 't' AND
  attname = 'c';
ANALYZE t; ANALYZE t (c); ALTER TABLE t
  ALTER COLUMN c SET STATISTICS 1000;
  ANALYZE t;
CREATE STATISTICS s_ab (dependencies,
  ndistinct, mcv) ON a, b FROM t; ANALYZE
  t;
```

Steering and experiments (per statement, never globally)

```
BEGIN; SET LOCAL work_mem = '256MB';
  EXPLAIN (ANALYZE, BUFFERS) ...;
  ROLLBACK;
SET enable_hashjoin = off; SET
  enable_seqscan = off; SET
  random_page_cost = 1.1; RESET ALL; SHOW
  shared_buffers;
```

Buffer pool

```
CREATE EXTENSION pg_buffercache; SELECT
  c.relname, count(*) FROM pg_buffercache
  b JOIN pg_class c ON b.relfilenode =
  pg_relation_filenode(c.oid) GROUP BY 1
  ORDER BY 2 DESC;
SELECT heap_blks_hit, heap_blks_read FROM
  pg_statio_user_tables WHERE relname =
  't'; -- h = hit / (hit + read)
```

Indexes

```
CREATE [UNIQUE] INDEX name ON t (a, b);
  ON t (lower(email)); ON t (user_id)
  WHERE status = 'open'; ON t (a) INCLUDE
  (b); ON t USING brin (ts); ON t USING
  hash (k); DROP INDEX name;
SELECT indexrelname, idx_scan FROM
  pg_stat_user_indexes WHERE relname =
  't'; \d t \timing on
```

Query skeleton (evaluation order in §1)

```
SELECT u.country, count(*) AS n,
  sum(o.amount) AS rev FROM orders o JOIN
  users u ON u.id = o.user_id WHERE
  o.status = 'paid' AND o.ts >= now() -
  interval '30 days' GROUP BY u.country
  HAVING count(*) > 100 ORDER BY rev DESC
  LIMIT 10;
```

Test data: one `random()` draw per row, several cuts, then ANALYZE:

```
INSERT INTO e (uid, kind) SELECT 1 +
  (random() * 999999)::int, CASE WHEN r <
  0.9 THEN 'view' WHEN r < 0.99 THEN
  'cart' ELSE 'buy' END FROM (SELECT
  random() AS r FROM generate_series(1,
  1000000)) g; ANALYZE e;
```

! `random()` inside each WHEN (a new draw per branch, wrong proportions). `EXPLAIN ANALYZE` on DML outside BEGIN/ROLLBACK. `CREATE STATISTICS` or `SET STATISTICS` without the `ANALYZE` after. `SET work_mem` globally. Forgetting `ANALYZE` after a bulk load.

9 Procedures for every question type

Storage estimate

record = columns +24, round to 8 ▸ $r = \lfloor 8168/(\text{record} + 4) \rfloor$ ▸ $B = \lceil N/r \rceil$, size $8B$ KB ▸ scan B; lookup without index $B/2$ present, B absent ▸ seconds = $B\times$ the device's per-page cost.

Index decision: does this index help?

h: bracket N between powers of F; lookup $h + 1$, range [FS] ▸ s from the predicate (rules [FS]) ▸ $k = sN$ ▸ pages touched: scattered $B(1-(1-s)^r)$, contiguous ≈ sB (read the correlation) ▸ index cost ≈ h+ leaves +4× pages touched, against Seq Scan B ▸ rule of thumb: plain index iff $s < 1/(4r)$, bitmap up to a few percent ▸ add the write price ▸ yes or no in one sentence.

External sort

runs = $\lceil B/M \rceil$ ▸ divide by $M - 1$ with a ceiling until 1 ▸ I/O = $2B \times$ (1 + merge passes) [FS] ▸ "what changes with M'": recompute, at most one pass fewer per doubling ▸ PostgreSQL: the `Sort Method` line, fix with `work_mem` ≥ the `Disk` figure.

Join cost comparison

data: $B_R, B_S, |R|, |S|, M, d$, indexes, order ▸ smaller input outside or build ▸ fits? ($M - 2$ for block NL, M for hash) ▸ every candidate from the table [FS], sort terms as full external sorts unless sorted ▸ minimum, then the trap answers (rows for pages, M for $M - 2$) ▸ when would the loser win.

Selectivity and estimated rows

per predicate: MCV frequency / $1/d$ / histogram share [FS] ▸ combine: AND product, OR inclusion–exclusion, NOT complement ▸ rows = sN ▸ join $|R||S|/\max(d)$; key–FK join = the FK side after its filters ▸ compare with actual: q-error; find the lowest wrong node ▸ name the failure (stale / correlated / skew) and the exact command.

Plan diagnosis

the five moves of §7 ▸ quote the node, est vs act, loops, Buffers, Sort Method or Batches ▸ mechanism: why the planner chose this given its estimate ▸ fix in order: ANALYZE → extended statistics → `work_mem` or index → rewrite ▸ re-run `EXPLAIN (ANALYZE, BUFFERS)` to verify.

Buffers line

hit + read = pages requested; compare with B (whole table? once or per loop?) ▸ h = hit/(hit + read) ▸ cold vs warm: `read` is not disk (OS cache) ▸ `temp` = spill, raise `work_mem` ▸ ringed? compare B with `shared_buffers`/4.

Replacement trace

draw the frames ▸ per request: hit (LRU: move to front; FIFO: nothing; clock: set the bit) or miss (victim by the rule, dirty ⇒ an extra write) ▸ count hits, misses, writes ▸ EAT ▸ flooding: W survives iff the scan is ringed and $W \le N - 32$.

Ticket or discussion answer

claim ▸ mechanism (subsystem, rule) ▸ a number or formula with units ▸ consequence and recommendation ▸ the tempting wrong answer. **Quick check**: letter + one number or sentence. **Situation**: data → formula → numbers with units → decision.